



I2C 接口配置 FLASH 应用解决方案

上海安路信息科技股份有限公司

APUG_063 (v1.4) 2025 年 09 月



目 录

目 录	I
1 概要	1
2 功能简述	2
2.1 功能特点	2
2.2 功能介绍	2
3 模块接口与功能介绍	3
3.1 端口说明	3
3.2 CMD 命令说明	4
3.2.1 CMD 0xC1	4
3.2.2 CMD 0xC2	5
3.2.3 CMD 0xC3	6
3.2.4 CMD 0xC4	7
3.2.5 CMD 0xC5	8
3.3 几种操作场景流程说明	9
3.3.1 正常数据读写流程	9
3.3.2 轮询中断流程	11
3.3.3 读取 Slave Device ID 流程	12
4 资源消耗与时序 Fmax	14
5 升级 FPGA Flash 操作说明	15
5.1 TD 软件设置	15
5.2 初始化	15
5.2.1 配置过程	15



5.2.2 嵌入式函数 API 说明	16
5.3 擦除 Flash.....	16
5.3.1 配置过程	16
5.3.2 嵌入式函数 API 说明	18
5.4 写 Flash.....	19
5.4.1 配置过程	19
5.4.2 嵌入式函数 API 说明	20
5.5 读 Flash.....	21
5.5.1 配置过程	21
5.5.2 嵌入式函数 API 说明	23
5.6 读 Flash 寄存器状态	23
5.6.1 配置过程	23
5.6.2 嵌入式函数 API 说明	26
5.7 读 Slave Device ID.....	26
5.7.1 配置过程	26
5.7.2 嵌入式函数 API 说明	27
5.8 读 Flash Device ID.....	27
5.8.1 配置过程	27
5.8.2 嵌入式函数 API 说明	29
5.9 重启 FPGA.....	29
5.9.1 配置过程	29
5.9.2 嵌入式函数 API 说明	30
6 工程说明	31
6.1 FPGA 工程说明.....	31
6.2 ARM 工程说明.....	31
6.3 工程目录	32



7 版本信息	33
免责声明	33



1 概要

本文档主要介绍 I2C（Inter-Integrated Circuit）总线接口在线配置 FPGA 器件 FLASH 的软硬件协议、结构及配置流程。文档中详细介绍了利用 CPU 配置 FPGA 器件的应用场景，为用户提供完整的操作流程说明。

2 功能简述

2.1 功能特点

- I2C 接口支持两种速度模式，分别是标准模式 100kbit/s 和快速模式 400kbit/s
- 具有可配置的 7bit I2C 总线从设备地址
- 具有可配置的 32bit Device ID
- 内置 512Byte 的缓存空间
- 支持 SPI 总线 4 种不同的时钟模式
- 可配置的 SPI 串行时钟频率
- 支持 I2C 主设备轮询中断
- 支持输出中断（低电平有效）
- 支持 SCL 和 SDA 输入滤波
- 可配置滤波的稳定时间

2.2 功能介绍

如图 2-1 所示，CPU 这边的 I2C 总线接口作为主设备，FPGA 这边的 I2C 总线接口作为从设备。在线配置 FPGA 器件由主设备发起，按照协议要求将数据传递到从设备，从设备进行缓存后与 Flash 进行数据通信，配置完成 Flash 后，可通过协议发起重启命令，整个 FPGA 配置过程完成。

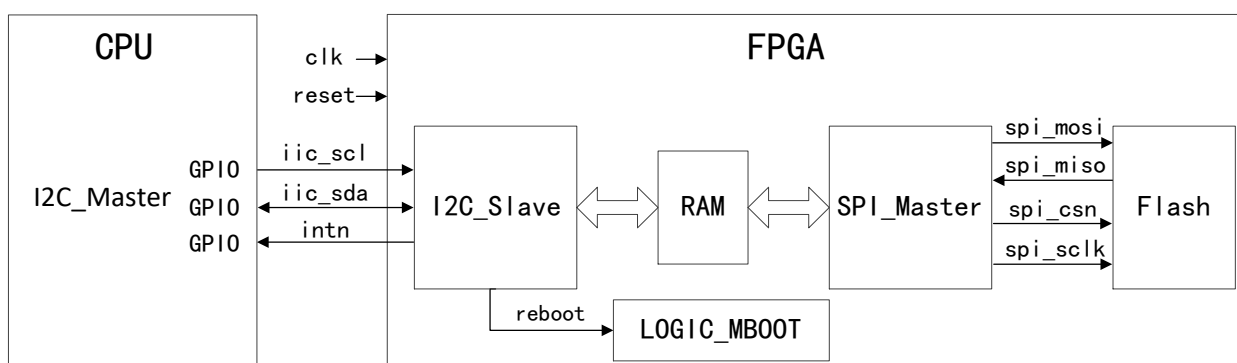


图 2-1 I2C 总线接口在线配置 FPGA 器件功能框图



3 模块接口与功能介绍

3.1 端口说明

本设计的主要端口，如表 3-1 所示。

表 3-1 端口信号表

信号	方向	功能说明
clk	in	系统时钟
reset	in	系统复位高电平有效
iic_scl	in	I2C 时钟
iic_sda	inout	I2C 数据
reboot	out	FPGA 器件重启信号
intn	out	中断信号
spi_miso	in	SPI 数据输入
spi_mosi	out	SPI 数据输出
spi_csn	out	SPI 片选
spi_sclk	out	SPI 时钟

除上述端口外，本设计还有可配置的参数，如表 3-2 所示。

表 3-2 参数设置表

参数名	可选值
DEVICE_ID	32 位 bit 数据
CLOCK_SEL	大于 0
I2C_SLAVE_ADDR	标准的 7bit 地址
STABLE_TIME	大于 0
CLK_FREQ	大于 0
HOLD_TIMING	大于 0

表 3-2 中，各个参数的功能如下：

- DEVICE_ID：作为 FPGA 器件的 32 位软 ID，用户可自定义；
- CLOCK_SEL：SPI 总线串行时钟频率的配置：

$$F_{spi_sclk} = \frac{F_{clk}}{2 * (CLOCK_SEL + 1)}$$

- I2C_SLAVE_ADDR：I2C Slave 的地址配置，支持标准的 7bit 地址；



- STABLE_TIME: 表示消抖稳定时间, 单位为 ns, 用于对 I2C 的两路信号进行输入滤波;
- CLK_FREQ: 系统时钟频率, 单位为 ns;
- HOLD_TIMING: 相对于系统时钟 clk, 将 iic_sda 信号进行打拍, 调整 iic_sda 的 hold 时间, 提高数据的采样率, 单位是拍。

3.2 CMD 命令说明

I2C 主设备到 I2C 从设备的协议命令, 如表 3-3 所示。

表 3-3 命令列表

命令	功能
0xC1	初始化多功能配置
0xC2	写数据模式
0xC3	清除中断
0xC4	设置读取中断模式
0xC5	设置读取 Device ID 模式

3.2.1 CMD 0xC1

当 I2C 主设备向 I2C 从设备发送命令 0xC1 后, I2C 主设备对 SPI 通信接口进行配置, 其配置过程如图 3-1 所示。



图 3-1 CMD 0xC1 配置过程

如图 3-1, 配置 SPI 接口的过程有以下步骤:

- (1) I2C 主设备向 I2C 从设备发送起始信号 (S);
- (2) I2C 主设备向 I2C 从设备发送设备地址 (Slave Address, 8bit 地址的高 7 位) 以及写信号 (W, 8bit 地址的最低位);
- (3) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (4) I2C 主设备向 I2C 从设备发送配置模式命令 (0xC1)
- (5) I2C 从设备向 I2C 主设备发送应答信号 (A);



- (6) I2C 主设备向 I2C 从设备发送 Configure SPI 数据;
- (7) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (8) I2C 主设备向 I2C 从设备发送停止信号 (P)。

其中, Configure SPI 的具体数据格式, 如表 3-4 所示。

表 3-4 Configure SPI bits

bit	5 至 7	4	3	2	1	0
Symbol	OTHERS	REBOOT	CSN	DIRECTION	CPHA	CPOL
说明	N/A	reboot	spi_csn	MSB	CPHA	CPOL
复位	1	1	1	0	0	0

注意: 当未完成对 Flash 的操作时, 请勿设置 reboot 信号为低电平。

表 3-4 中 MSB 表示 SPI 数据的对齐位, 当设置为 1 时, 表示低位对齐; 设置为 0 时, 表示高位对齐。

时钟极性 (CPOL) 和时钟相位 (CPHA) 决定了 SPI 时钟信号的哪个沿将用于驱动或接收数据。SPI 主设备和从设备必须使用相同的时钟极性和时钟相位模式进行通信。本设计支持所有四种 SPI 时钟极性和时钟相位模式。

- Mode0: CPOL=0, CPHA=0
- Mode1: CPOL=0, CPHA=1
- Mode2: CPOL=1, CPHA=0
- Mode3: CPOL=1, CPHA=1

时钟极性 CPOL 是用来配置 SCLK 的电平处于哪种状态时是空闲态或者有效态, 时钟相位 CPHA 是用来配置数据采样是在第几个边沿进行。

- CPOL=0, 表示 SCLK=0 时是空闲态, SCLK=1 时是有效态
- CPOL=1, 表示 SCLK=1 时是空闲态, SCLK=0 时是有效态
- CPHA=0, 表示在每个周期第 1 个边沿进行数据采样, 在第 2 个边沿进行数据发送
- CPHA=1, 表示在每个周期第 2 个边沿进行数据采样, 在第 1 个边沿进行数据发送

3.2.2 CMD 0xC2

当 I2C 主设备向 I2C 从设备发送命令 0xC2 后, I2C 主设备开始将数据写入 I2C 从设备, 写数据过程如图 3-2 所示。

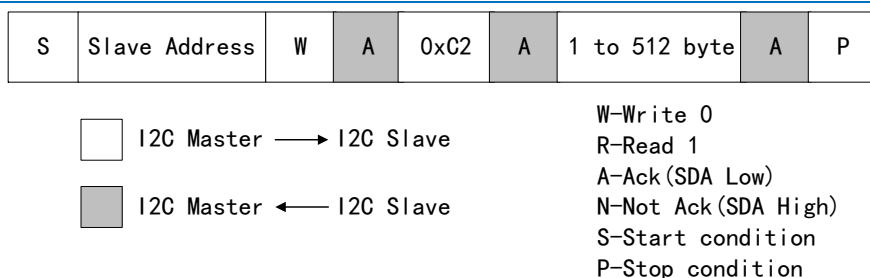


图 3-2 CMD 0xC2 配置过程

如图 3-2，写数据的过程有以下步骤：

- (1) I2C 主设备向 I2C 从设备发送起始信号 (S)；
- (2) I2C 主设备向 I2C 从设备发送设备地址 (Slave Address, 8bit 地址的高 7 位) 以及写信号 (W, 8bit 地址的最低位)；
- (3) I2C 从设备向 I2C 主设备发送应答信号 (A)；
- (4) I2C 主设备向 I2C 从设备发送写数据模式命令 (0xC2)；
- (5) I2C 从设备向 I2C 主设备发送应答信号 (A)；
- (6) I2C 主设备向 I2C 从设备发送数据，从设备将接收到的数据暂存到数据缓冲区中；
- (7) I2C 从设备向 I2C 主设备发送应答信号 (A)；
- (8) I2C 主设备向 I2C 从设备发送停止信号 (P)。

3.2.3 CMD 0xC3

当 I2C 主设备发送命令 0xC3 后，进行清除中断操作，其过程如图 3-3 所示。



图 3-3 CMD 0xC3 配置过程

如图 3-3，清除中断操作有以下步骤：

- (1) I2C 主设备向 I2C 从设备发送起始信号 (S)；
- (2) I2C 主设备向 I2C 从设备发送设备地址 (Slave Address, 8bit 地址的高 7 位) 以及写信号 (W, 8bit 地址的最低位)；

- (3) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (4) I2C 主设备向 I2C 从设备发送清除中断命令 (0xC3);
- (5) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (6) I2C 主设备向 I2C 从设备发送停止信号 (P)。

当 SPI 通信完成后, 产生中断有效信号, 此时, I2C 主设备无法读取 FLASH 中的数据, 需要先进行清除中断操作, I2C 主设备才可以读取到 FLASH 中的数据。

3.2.4 CMD 0xC4

当 I2C 主设备发送命令 0xC4 后, I2C 主设备可以读取中断信号。设置读取中断模式命令的过程, 如图 3-4 所示。



图 3-4 CMD 0xC4 配置过程

如图 3-4, 设置读取中断模式有以下步骤:

- (1) I2C 主设备向 I2C 从设备发送起始信号 (S);
- (2) I2C 主设备向 I2C 从设备发送设备地址 (Slave Address, 8bit 地址的高 7 位) 以及写信号 (W, 8bit 地址的最低位);
- (3) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (4) I2C 主设备向 I2C 从设备发送设置读取中断模式命令 (0xC4);
- (5) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (6) I2C 主设备向 I2C 从设备发送停止信号 (P)。

I2C 主设备读取数据的过程, 如图 3-5 所示。

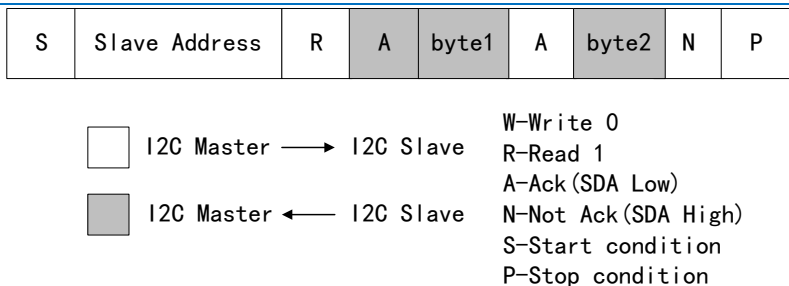


图 3-5 读取缓冲区数据过程

如图 3-5，读中断有以下步骤：

- (1) I2C 主设备向 I2C 从设备发送起始信号 (S)；
- (2) I2C 主设备向 I2C 从设备发送设备地址 (Slave Address, 8bit 地址的高 7 位) 以及读信号 (R, 8bit 地址的最低位)；
- (3) I2C 从设备向 I2C 主设备发送应答信号 (A)；
- (4) I2C 主设备开始读取数据，一般读取两个字节的的数据，然后判断读取数据的最低位是否为低电平，如果是，则表示中断信号产生。读取完数据后，I2C 主设备向从设备发送应答信号 (A)；
- (5) I2C 主设备向 I2C 从设备发送停止信号 (P)。

注意，当 I2C 回读到“0”时，表示中断产生，如果继续进行读取操作，需要清除中断。

3.2.5 CMD 0xC5

当 I2C 主设备发送命令 0xC5 后，I2C 主设备至少读取四个字节的的数据，读取的顺序是从低字节数据开始。例如，设备 ID 设置为 32'h87654321，I2C 主设备读取的顺序依次为 21 43 65 87。如果读取大于四个字节的的数据，就循环发送数据。其中，I2C 主设备发送命令 0xC5 的过程，如图 3-6 所示。



图 3-6 CMD 0xC5 配置过程

如图 3-6，设置读取 Device ID 模式有以下步骤：

- (1) I2C 主设备向 I2C 从设备发送起始信号 (S)；
- (2) I2C 主设备向 I2C 从设备发送设备地址 (Slave Address, 8bit 地址的高 7 位) 以及写信号 (W, 8bit 地址的最低位)；

- (3) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (4) I2C 主设备向 I2C 从设备发送设置读取 Device ID 模式命令 (0xC5);
- (5) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (6) I2C 主设备向 I2C 从设备发送停止信号 (P)。

I2C 主设备读取 5 个字节的 Device ID 的过程，如图 3-2-7 所示。

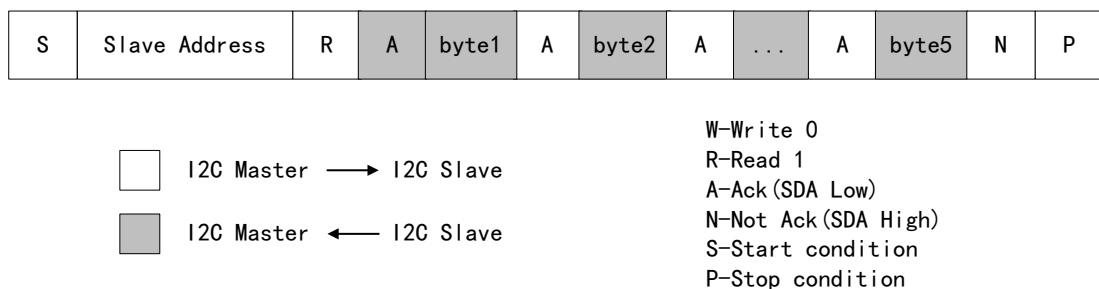


图 3-7 读取设备 ID 过程

如图 3-7，I2C 读取设备 ID 有以下步骤：

- (1) I2C 主设备向 I2C 从设备发送起始信号 (S);
- (2) I2C 主设备向 I2C 从设备发送设备地址 (Slave Address, 8bit 地址的高 7 位) 以及读信号 (R, 8bit 地址的最低位);
- (3) I2C 从设备向 I2C 主设备发送应答信号 (A);
- (4) I2C 主设备开始读取数据，每读取一个字节数据，I2C 主设备向 I2C 从设备发送应答信号 (A);
- (5) I2C 主设备读取结束时，I2C 主设备向 I2C 从设备发送停止信号 (P)。

3.3 几种操作场景流程说明

3.3.1 正常数据读写流程

I2C 主设备向 Flash 读写数据的流程图，如图 3-8，3-9 所示。

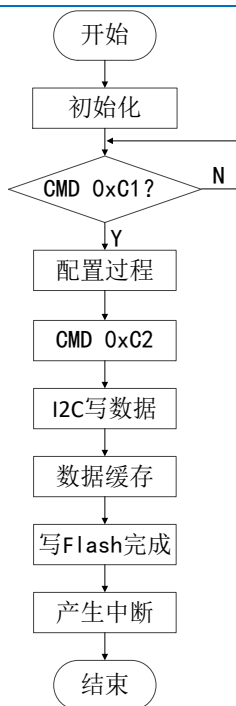


图 3-8 正常数据写流程图

如图 3-8，首先对模块进行初始化，发送配置命令 0xC1，接收到该命令后按照配置的值对 SPI 接口进行配置，完成后发送 0xC2 命令，接收到该命令后将 I2C 数据缓存到 RAM 中，然后开始向 Flash 写数据，写 Flash 完成后，产生中断信号，最后结束 I2C 主设备向 Flash 写数据。

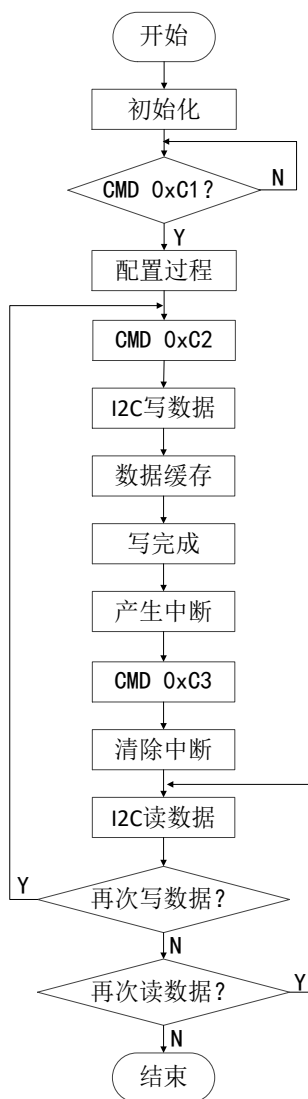


图 3-9 正常数据先写后读流程图

如图 3-9，首先对模块进行初始化，发送配置命令 0xC1，接收到该命令后按照配置的值对 SPI 接口进行配置，完成后发送 0xC2 命令，接收到该命令后将 I2C 数据缓存到 RAM 中，然后开始向 Flash 写数据，写 Flash 完成后，产生中断信号，此时状态发送 0xC3 命令来清除中断，中断清除后，I2C 进行读数据操作，读操作完成后，最后等待 I2C 是否再进行读写数据操作。

3.3.2 轮询中断流程

I2C 主设备轮询中断的流程图，如图 3-10 所示。图中首先对模块进行初始化，发送配置命令 0xC1，接收到该命令后按照配置的值对 SPI 接口进行配置，完成后发送 0xC2 命令，接收到该命令后 I2C 向 Flash 写数据，写完数据后发送 0xC4 命令，接收到命令后 I2C 开始读中断，若没有读取到中断信号，则继续读取中断信号，若读取到中断信号，则等待 I2C 是否再读中断。

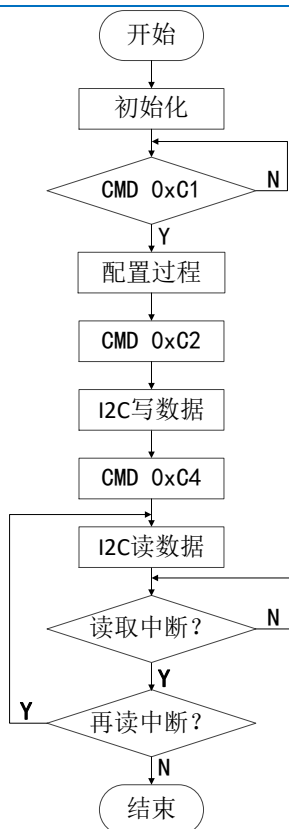


图 3-10 轮询中断流程图

3.3.3 读取 Slave Device ID 流程

I2C 主设备读取 Slave Device ID 的流程图，如图 3-11 所示。

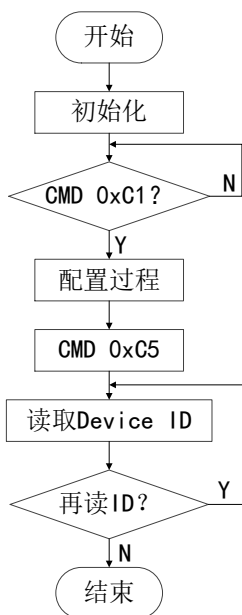


图 3-11 读取 Slave Device ID 流程图



如图 3-11，首先对模块进行初始化，发送配置命令 0xC1，接收到该命令后按照配置的值对 SPI 接口进行配置，完成后发送 0xC5 命令，接收到该命令后 I2C 读取 Slave Device ID，读取至少四个字节的 ID 后，等待 I2C 是否再读 ID。



4 资源消耗与时序 Fmax

使用 TD 软件版本为 TD5. 6. 2_71036, 选择 EF2M45 系列芯片进行软件验证, 消耗的资源与时序结果, 如表 4-1 所示。

表 4-1 资源消耗与时序 Fmax

资源	类型	数值
最大频率		79MHz
使用资源	LUT	330
	REG	158
	PLL	0
	IO	13
	BRAM	1
	DSP	0

5 升级 FPGA Flash 操作说明

5.1 TD 软件设置

打开 TD 工程，点击 Process→Properties→Generate Bitstream→Control Option（或者在 FPGA Flow-Phy Opt 中，右击 Generate Bitstream，点击 Properties→Generate Bitstream→Control Option），将 Persist_bit 设置成 0，进入用户模式，此时对 Flash 开启访问权限，设置界面如图 5-1 所示。

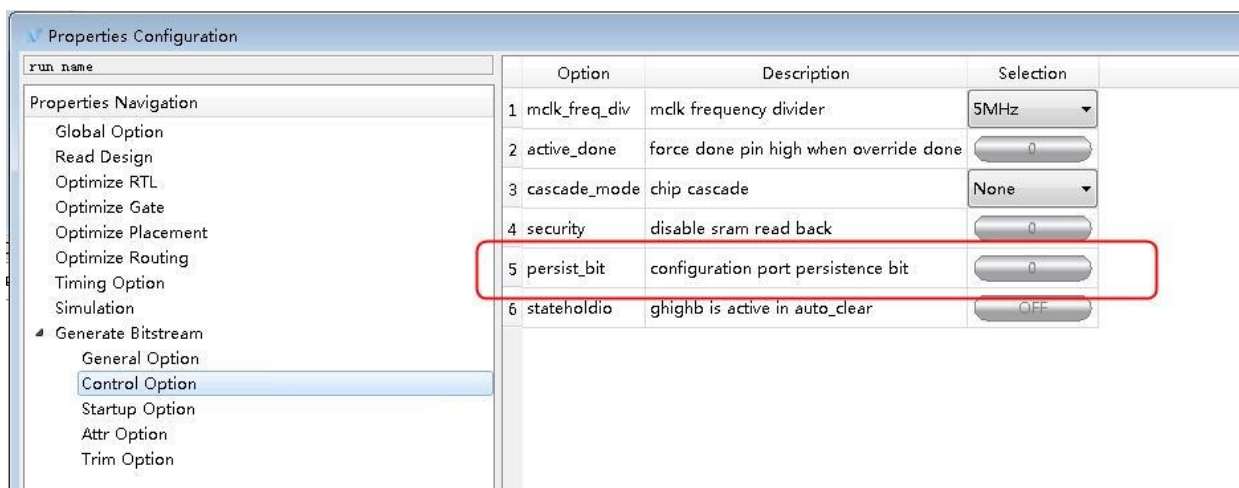


图 5-1 TD 软件设置界面图

5.2 初始化

5.2.1 配置过程

I2C 主设备向 I2C 从设备发送 0xC1 命令后，I2C 主设备对 SPI 接口进行配置。I2C 主设备初始化过程示意图，如图 5-2 所示。

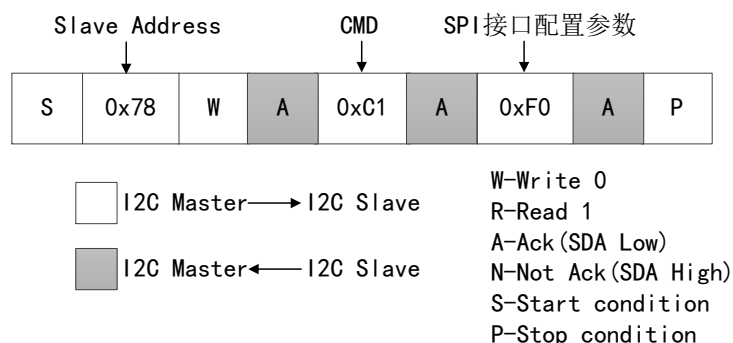


图 5-2 初始化过程时序示意图

图 5-2 中，假设 I2C 从设备地址设置为 0x78，SPI 接口配置参数为 0xF0（OTHERS 以及 REBOOT 设置



为 1，CSN、DIRECTION、CPHA 以及 CPOL 设置为 0，在上升沿采样）。

I2C 主设备初始化过程的时序图，如图 5-3 所示。

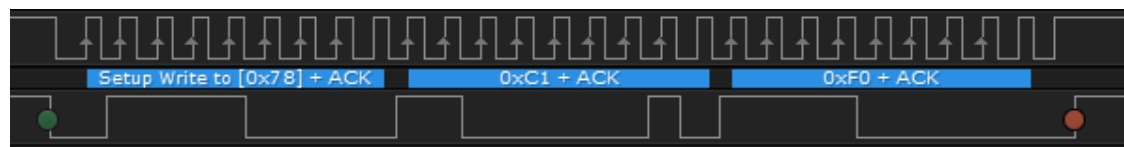


图 5-3 初始化过程时序图

5.2.2 嵌入式函数 API 说明

```
void CfgCMD01(unsigned char cfg_value)
```

CfgCMD01 函数，cfg_value 为待配置值。

5.3 擦除 Flash

5.3.1 配置过程

擦除 Flash，需要经过写使能、写擦除地址、读中断、清除中断、延时等待或者读 Flash 状态寄存器五个步骤，假设 I2C 从设备地址设置为 0x78。

第一步：写使能。I2C 主设备向 I2C 从设备发送 0xC2 命令后，再发送 Flash 的写使能命令 0x06，其时序示意图，如图 5-4 所示。

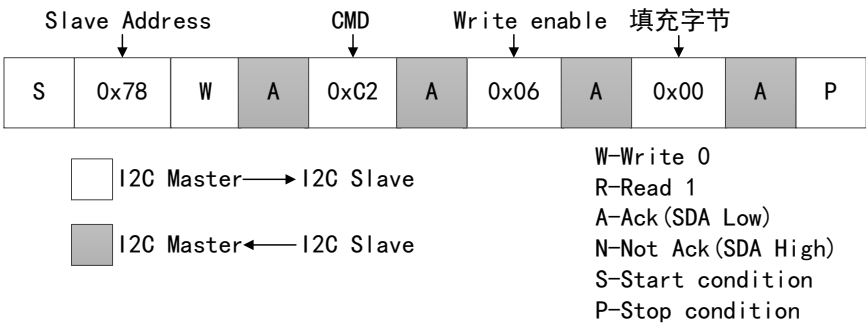


图 5-4 I2C 主设备写使能时序示意图

在图 5-4 中，因协议要求，故应额外填充至少一个字节的 0。I2C 主设备写使能时序图，如图 5-5 所示。



图 5-5 I2C 主设备写使能时序图



第二步：写擦除地址，I2C 主设备写擦除地址的时序示意图，如图 5-6 所示。

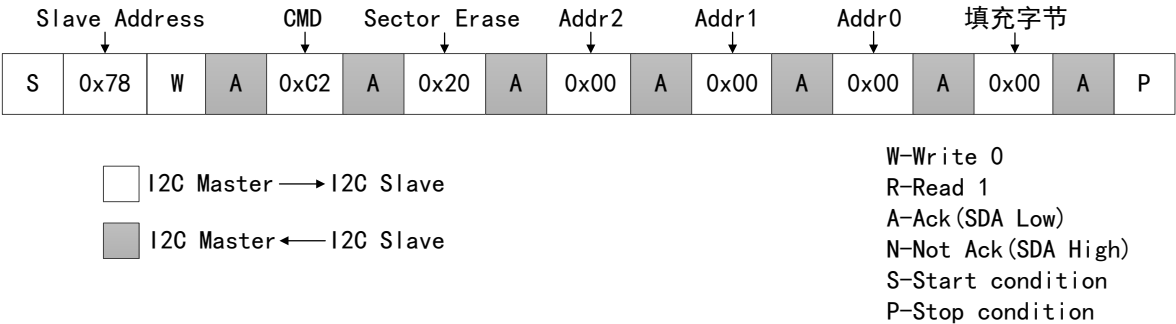


图 5-6 I2C 主设备写擦除地址时序示意图

在图 5-6 中，Flash 的 Sector Erase 命令为 0x20，三个单元存储地址 Addr2，Addr1，Addr0 都为 0x00。其中，Addr2 表示 Flash 24bit 地址的高 8bit，即 Addr[23:16]；Addr1 表示 Flash 24bit 地址的中 8bit，即 Addr[15:8]；Addr0 表示 Flash 24bit 地址的低 8bit，即 Addr[7:0]。因协议要求，故应额外填充至少一个字节 0。

I2C 主设备写擦除地址的时序图，如图 5-7 所示。

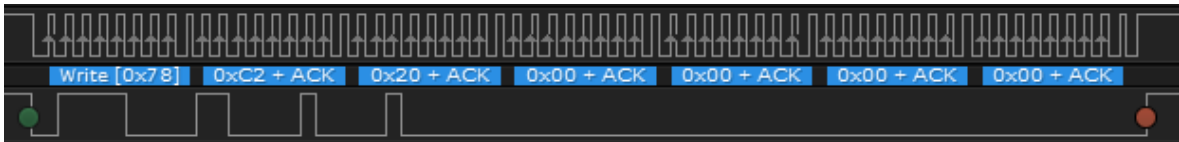


图 5-7 I2C 主设备写擦除地址时序图

第三步，读中断信号。I2C 主设备读中断整个的时序示意图，如图 5-8 所示。

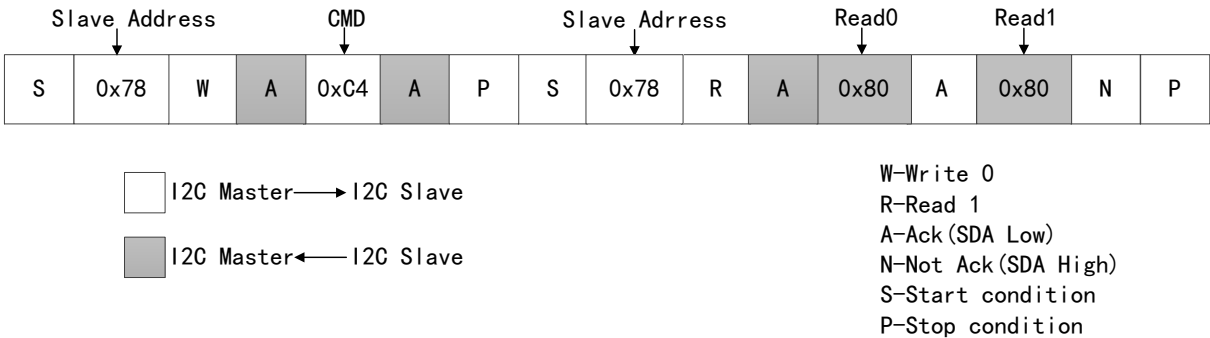


图 5-8 I2C 主设备读中断的整个时序示意图

图 5-8 中，I2C 主设备读取到两个字节的数 Read0 和 Read1 都为 0x80，且最低位的数据为低电平，表示中断信号产生。I2C 主设备读中断的整个时序图，如图 5-9 所示。

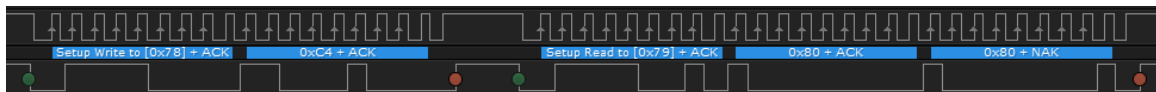


图 5-9 I2C 主设备读中断的整个时序图

从图 5-9 中可看出，中断信号已产生，因此需要 I2C 主设备清除中断信号。

第四步，清除中断。I2C 主设备清除中断信号的时序示意图，如图 5-10 所示。

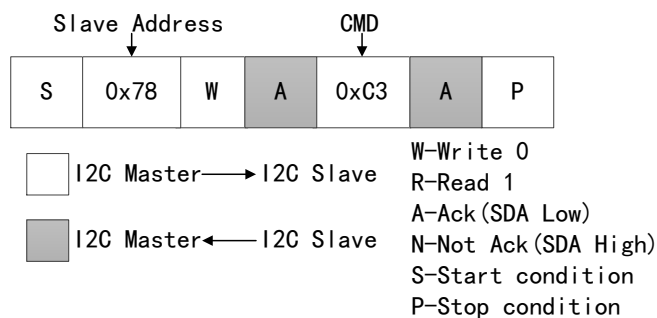


图 5-10 I2C 主设备清除中断信号的时序示意图

I2C 主设备清除中断信号的时序图，如图 5-11 所示。



图 5-11 I2C 主设备清除中断信号的时序图

第五步，延时，等待 Flash 擦除操作完成（300ms）或者通过读 Flash 状态寄存器来判断 Flash 是否擦除完毕（详细见 5.6 节）。

5.3.2 嵌入式函数 API 说明

```
void CfgCMDC2_Wren()
```

Wren 操作函数；

```
void CfgCMDC2_ChipErase()
```

ChipErase 操作函数，若 dualboot 功能，推荐使用 SectorErase；

```
void CfgCMDC2_SectorErase(unsigned char addr0,unsigned char addr1,unsigned char addr2)
```

Sector Erase 操作函数，addr0，addr1 和 addr2 为 Flash 地址；

5.4 写 Flash

5.4.1 配置过程

I2C 主设备写 Flash，需要经过写使能、写数据、延时等待以及轮询中断四个步骤，I2C 从设备地址设置为 0x78，下文给出具体介绍。

第一步：写使能。I2C 主设备发送 0xC2 命令后，再发送 Flash 的写使能命令 0x06，其时序示意图，如图 5-12 所示。

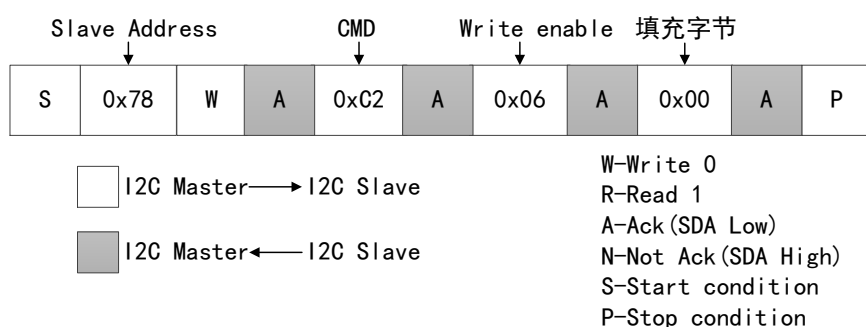


图 5-12 I2C 主设备写使能时序示意图

I2C 主设备写使能时序图，如图 5-13 所示。



图 5-13 I2C 主设备写使能时序图

第二步：写数据。I2C 主设备向 Flash 里的 Addr 写数据，每次最多写入 256 个数据。I2C 主设备向 Addr 写数据的时序示意图，如图 5-14 所示。

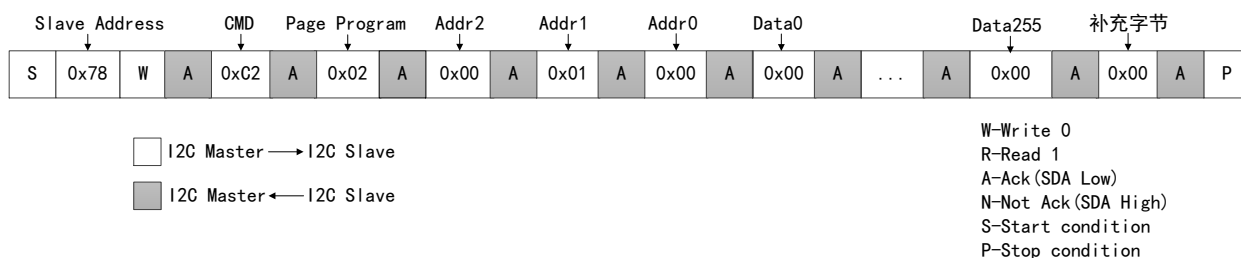


图 5-14 I2C 主设备写数据时序示意图

图 5-14 中，Flash 的 Page Program 为 0x02，三个单元存储地址 Addr2, Addr1, Addr0 分别为 0x00, 0x01, 0x00。其中，Addr2 表示 Flash 24bit 地址的高 8bit，即 Addr[23:16]；Addr1 表示 Flash 24bit 地址的中 8bit，即 Addr[15:8]；Addr0 表示 Flash 24bit 地址的低 8bit，即 Addr[7:0]。Data0 到



Data255 设置为 0x00。其中，Data0 表示 I2C 主设备发送给 Flash 的第 1 个数据，Data255 表示 I2C 主设备发送给 Flash 的第 256 个数据，以此类推。

I2C 主设备向 Addr 写数据的时序图，如图 5-15 所示。

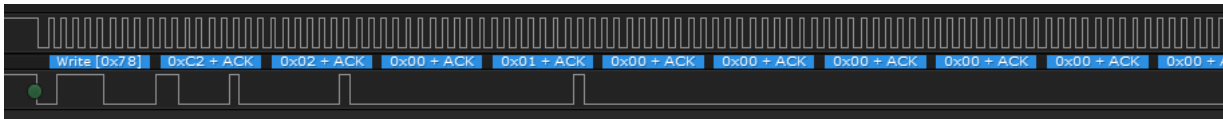


图 5-15 I2C 主设备写数据时序图

第三步，增加延时，等待 Flash 写操作完成（5ms）（或者通过读 Flash 寄存器状态来判断 Flash 写操作是否完成）。

第四步，读中断。I2C 主设备向 I2C 从设备发送 0xC4 命令后，I2C 主设备可以读取中断信号，假设 I2C 从设备地址设置为 0x78，I2C 主设备读中断的整个时序示意图，如图 5-16 所示。

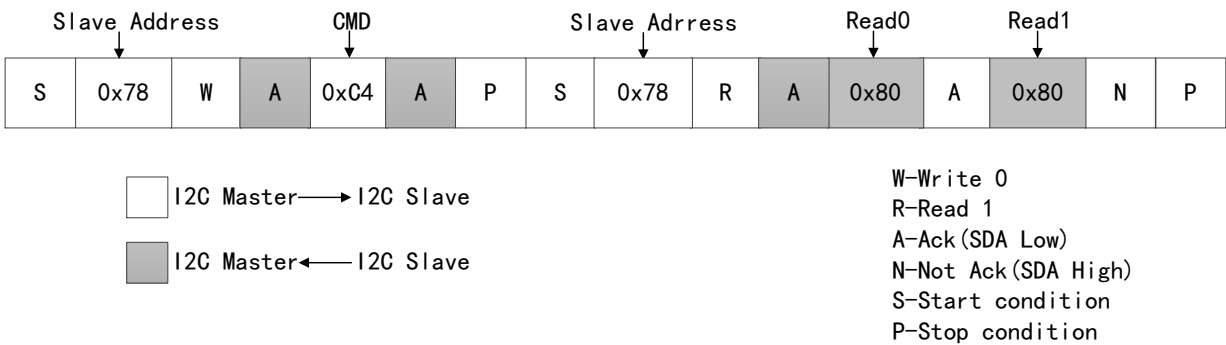


图 5-16 I2C 主设备读中断的整个时序示意图

图 5-4-5 中，I2C 主设备读取到两个字节的数 Read0 和 Read1 都为 0x80，且数据的最低位为低电平，表示中断信号产生。

I2C 主设备读中断的整个时序图，如图 5-17 所示。



图 5-17 I2C 主设备读中断的整个时序图

5. 4. 2 嵌入式函数 API 说明

Wren 操作函数：

```
void CfgCMD2_Wren()
```



Flash 写操作函数:

```
void CfgCMDC2_WrData(unsigned char addr0, unsigned char addr1, unsigned char addr2)
```

其中, addr0, addr1 和 addr2 为写地址。

5.5 读 Flash

5.5.1 配置过程

I2C 主设备读 Flash 时, 需要经过写数据、读中断、清除中断、读 Flash 数据四个步骤, 假设 Slave Address 为 0x78。

第一步, 写数据。I2C 主设备发送完 0xC2 命令后, 再发送 Flash 的 Read Date 命令 0x03, 即可将 Flash 中的 256 个数据转存到 RAM 中, 其时序示意图, 如图 5-18 所示。

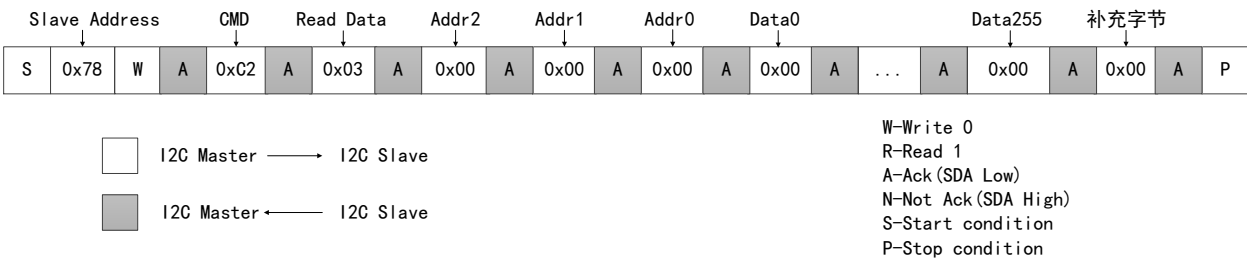


图 5-18 I2C 主设备将 Flash 数据转存到 RAM 中的时序示意图

图 5-18 中, Addr2, Addr1, Addr0 都为 0x00, 其中, Addr2 表示 Flash 24bit 地址的高 8bit, 即 Addr[23:16]; Addr1 表示 Flash 24bit 地址的中 8bit, 即 Addr[15:8]; Addr0 表示 Flash 24bit 地址的低 8bit, 即 Addr[7:0]。Data0 到 Data255 默认设置为 0x00。

I2C 主设备将 Flash 数据转存到 RAM 中的时序图, 如图 5-19 所示。

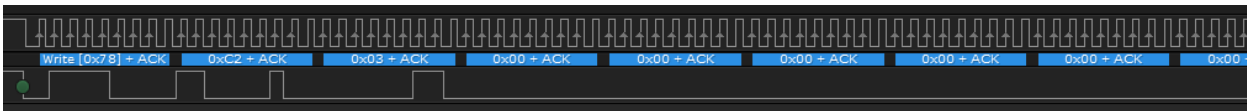


图 5-19 I2C 主设备将 Flash 数据转存到 RAM 中的时序图

第二步, 读中断。I2C 主设备发送 0xC4 命令后, 读取中断信号, 其时序示意图, 如图 5-20 所示。

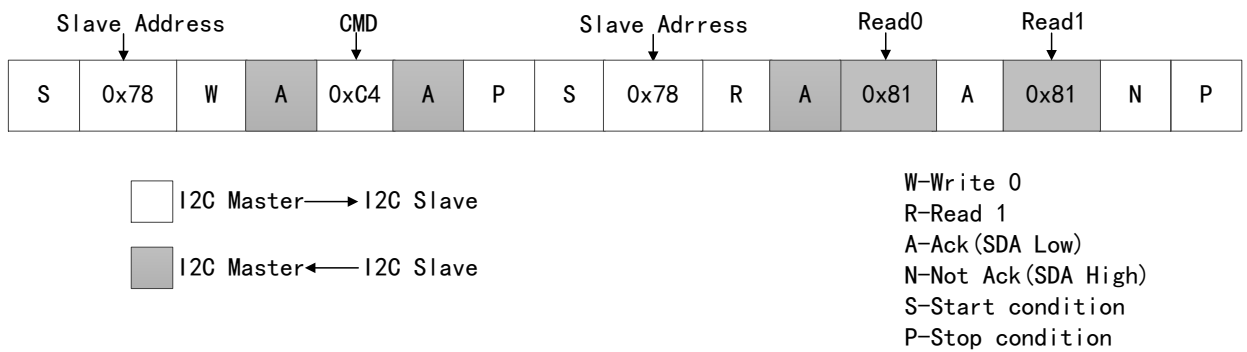


图 5-20 I2C 主设备读中断的整个时序示意图

图 5-20 中，I2C 主设备读取到两个字节的 **Read0** 和 **Read1** 都为 0x81，且数据的最低位为高电平，表示还未产生中断信号，此时 I2C 主设备读中断的整个时序图，如图 5-21 所示。



图 5-21 I2C 主设备读中断的整个时序图

由于中断信号还没有产生，因此 I2C 主设备再次发起 0xC4 命令，继续读取中断信号，此时 I2C 主设备读中断的整个时序图，如图 5-22 所示。

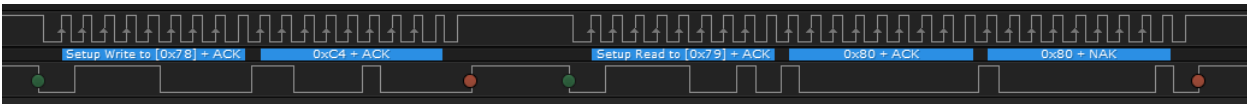


图 5-22 I2C 主设备读中断的整个时序图

图 5-22 中，I2C 主设备读取到的数据最低位都为低电平，表示中断信号已产生，因此需要 I2C 主设备清除中断信号。

第三步，清除中断。I2C 主设备发送 0xC3 命令清除中断的时序示意图，如图 5-23 所示。

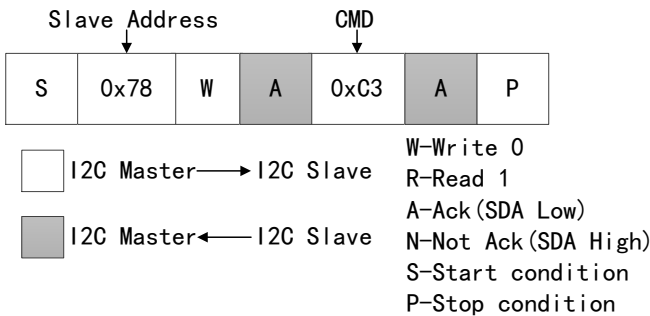


图 5-23 I2C 主设备清除中断信号的时序示意图

I2C 主设备清除中断信号的时序图，如图 5-24 所示。



图 5-24 I2C 主设备清除中断信号的时序图

第四步，读 Flash 数据。I2C 主设备将 RAM 中的数据读取到 I2C 接口的时序示意图，如图 5-25 所示。

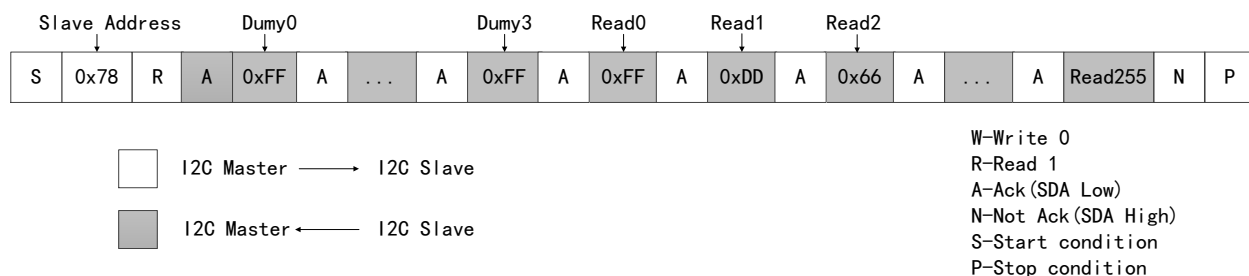


图 5-25 从 RAM 中读取数据到 I2C 接口时序示意图

图 5-25 中，Slave Address 为 0x78，Dummy0 到 Dummy3 为无效数据，Read0、Read1、Read2 分别为 0xFF，0xDD，0x66，Read n 表示 FPGA 回读给 CPU 的第 n 个数据，结束标识需要 CPU 侧的 I2C 主设备主动发 Stop 来终止读数据。

I2C 主设备将 RAM 中的数据读取到 I2C 接口的时序图，如图 5-26 所示。



图 5-26 从 RAM 中读取数据到 I2C 接口时序图

5.5.2 嵌入式函数 API 说明

```
void CfgCMD2_RdData(unsigned char addr0,unsigned char addr1,unsigned char addr2)
```

Flash 读操作，addr0，addr1 和 addr2 为读地址。

5.6 读 Flash 寄存器状态

5.6.1 配置过程

I2C 主设备读 Flash 寄存器状态需要经过 I2C 主设备发送读 Flash 状态寄存器命令 0x05、读中断、清除中断、读取出状态寄存器中的数据这四个步骤，假设 I2C 从设备地址为 0x78。

第一步，I2C 主设备发送读 Flash 状态寄存器命令 0x05，其时序示意图，如图 5-27 所示。



图 5-27 I2C 主设备发送读 Flash 状态寄存器命令的时序示意图

I2C 主设备发送读 Flash 状态寄存器命令的时序图，如图 5-28 所示。



图 5-28 I2C 主设备发送读 Flash 状态寄存器命令的时序示意图

第二步，读中断信号。I2C 主设备读中断整个的时序示意图，如图 5-29 所示。

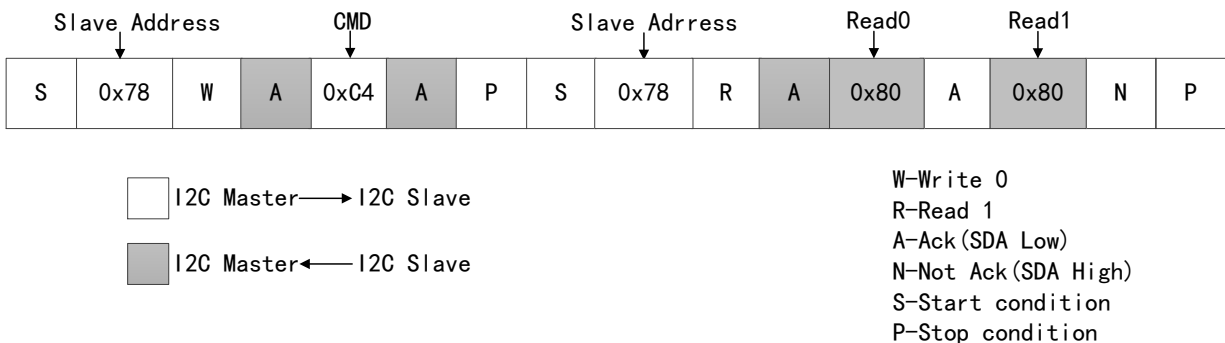


图 5-29 I2C 主设备读中断的整个时序示意图

图 5-29 中，I2C 主设备读取到两个字节的 **Read0** 和 **Read1** 都为 0x80，且数据的最低位为低电平，表示中断信号产生。I2C 主设备读中断的整个时序图，如图 5-30 所示。



图 5-30 I2C 主设备读中断的整个时序图

从图 5-30 中可看出，中断信号已产生，因此需要 I2C 主设备清除中断信号。

第三步，清除中断。I2C 主设备清除中断信号的时序示意图，如图 5-31 所示。

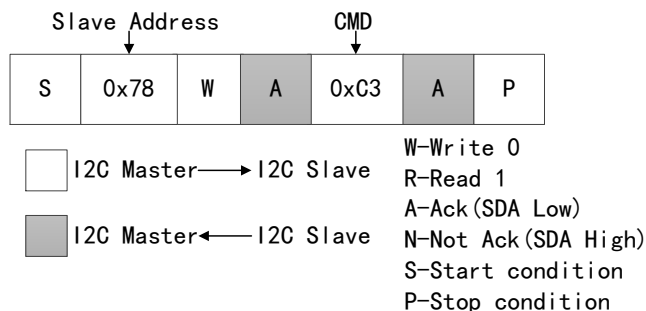


图 5-31 I2C 主设备清除中断信号的时序示意图

I2C 主设备清除中断信号的时序图，如图 5-32 所示。



图 5-32 I2C 主设备清除中断信号的时序图

第四步，I2C 主设备读取状态寄存器数据的时序示意图，如图 5-33 所示。



图 5-33 I2C 主设备读取状态寄存器数据的时序示意图

在图 5-33 中，Dummy0 为无效数据，其值为 0xFF；Read0、Read1、Read2 为有效数据，其值为 0x03。当有效数据的最低 bit 是 1 时，表示 Flash 处于忙状态；最低 bit 是 0 时，表示 Flash 处于空闲状态。由于图中的有效数据 0x03 最低 bit 为 1，表示 Flash 处于忙状态。因此通过读取有效数据的最低 bit 来判断 Flash 的状态，可以节省不必要的等待时间。

I2C 主设备读 Flash 状态寄存器数据的时序图，如图 5-34 所示。



图 5-34 I2C 主设备读 Flash 状态寄存器数据的时序图

5.6.2 嵌入式函数 API 说明

```
void CfgCMDC5_isBusy(void)
```

读取 Flash WIP 位，判断是否处于 page program/ sector erase 状态。

5.7 读 Slave Device ID

5.7.1 配置过程

I2C 主设备发送 0xC5 命令后，读取 Slave Device ID 的时序示意图，如图 5-35，5-36 所示。

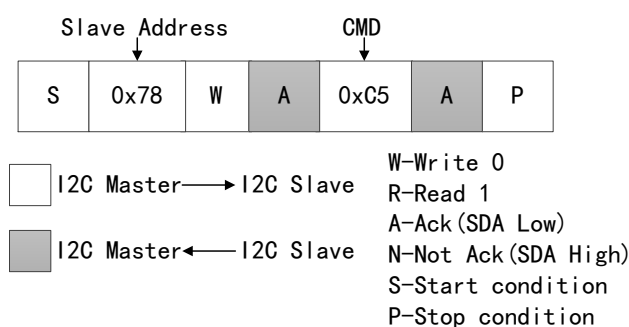


图 5-35 I2C 主设备发送 0xC5 命令的时序示意图

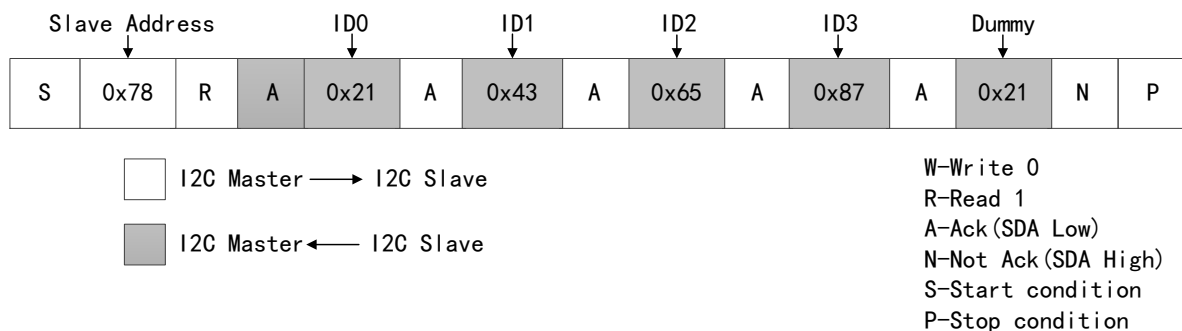


图 5-36 I2C 主设备读取 Slave Device ID 的时序示意图

当设备 ID 设置为 32'h87654321，I2C 主设备发送 0xC5 命令后，I2C 主设备向 I2C 从设备读取至少四个字节的数，读到的依次是低字节数据，对应地 ID0，ID1，ID2，ID3 分别为 0x21，0x43，0x65，0x87。如果读取大于四个字节数据，会循环向 I2C 主设备发送，例如图 5-7-2 中的 Dummy 为 0x21。

I2C 主设备读取 Slave Device ID 的时序图，如图 5-37 所示。



图 5-37 I2C 主设备读取 Slave Device ID 的时序图

5.7.2 嵌入式函数 API 说明

```
void CfgCMDC5_RdSlaveDeviceID(void)
```

读取 I2C Slave Device ID，可在 HDL 中自定义。

5.8 读 Flash Device ID

5.8.1 配置过程

I2C 主设备读取 Flash Device ID 时，需要经过写数据、读中断、清除中断、读 Flash Device ID 四个步骤。

第一步，写数据。I2C 主设备发送 0xC2 命令后，再发送读取 Flash Device ID 的命令 0x9F，其时序示意图，如图 5-38 所示。

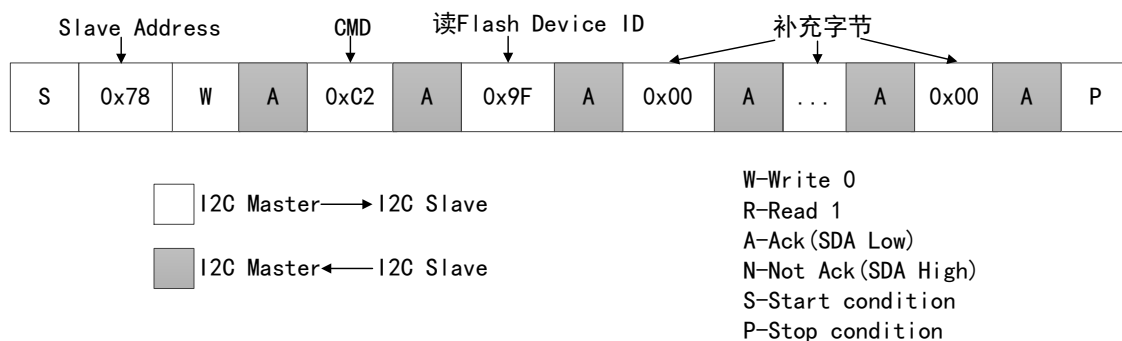


图 5-38 I2C 主设备发送读 Flash Device ID 命令的时序示意图

图 5-38 中，因协议要求，需要补充 4 个补充字节。I2C 主设备发送读取 Flash Device ID 命令的时序图，如图 5-39 所示。

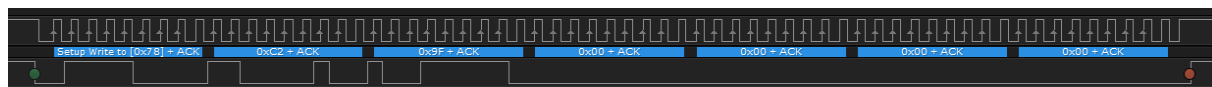


图 5-39 I2C 主设备发送读 Flash Device ID 命令的时序图

第二步，读中断信号。I2C 主设备读中断整个的时序示意图，如图 5-40 所示。

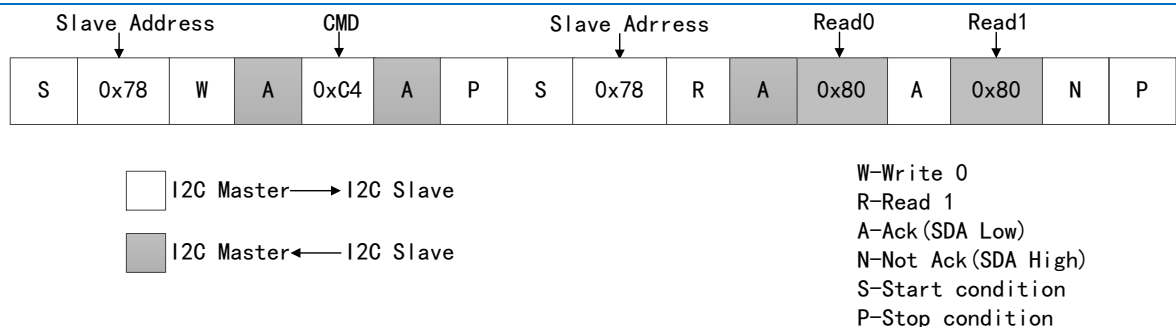


图 5-40 I2C 主设备读中断的整个时序示意图

图 5-40 中，I2C 主设备读取到两个字节的 **Read0** 和 **Read1** 都为 0x80，且数据的最低位为低电平，表示中断信号产生。I2C 主设备读中断的整个时序图，如图 5-41 所示。



图 5-41 I2C 主设备读中断的整个时序图

从图 5-41 中可看出，读取到的数据最低位为低电平，表示中断信号已产生，因此需要 I2C 主设备清除中断信号。

第三步，清除中断。I2C 主设备清除中断信号的时序示意图，如图 5-42 所示。

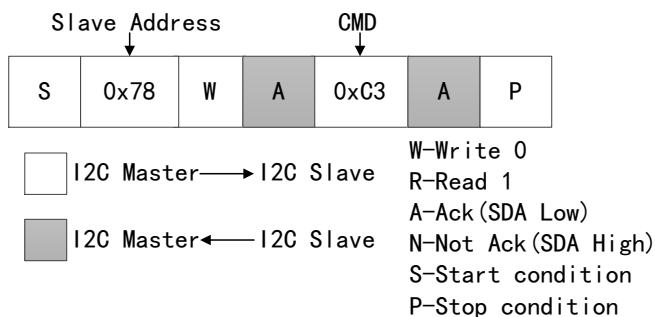


图 5-42 I2C 主设备清除中断信号的时序示意图

I2C 主设备清除中断信号的时序图，如图 5-43 所示。



图 5-43 I2C 主设备清除中断信号的时序图

第四步，I2C 主设备读取 Flash Device ID 的时序示意图，如图 5-44 所示。

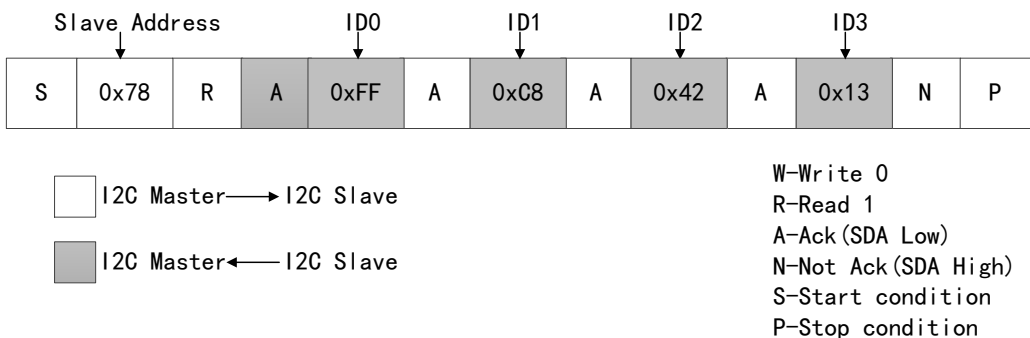


图 5-44 I2C 主设备读取 Flash Device ID 的时序示意图

当 Flash 设备 ID 设置为 32'hFFC84213，I2C 主设备发送 0xC5 命令后，I2C 主设备向 I2C 从设备读取至少四个字节的 ID 数据，读到的依次是低字节数据，对应地 ID0，ID1，ID2，ID3 分别为 0x13，0x42，0xC8，0xFF。

I2C 主设备读 Flash Device ID 的时序图，如图 5-45 所示。

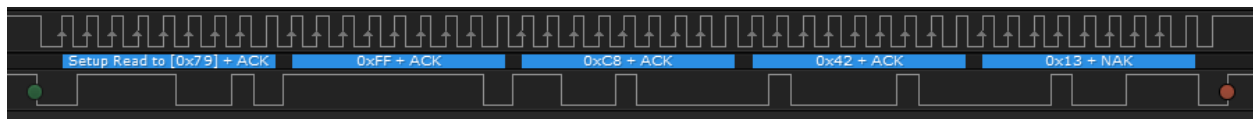


图 5-45 I2C 主设备读取 Flash Device ID 的时序图

5.8.2 嵌入式函数 API 说明

```
void CfgCMD2_RdFlashID( )
```

通过 Flash 0x9F 命令，读取 Flash 器件中的 ID。

5.9 重启 FPGA

5.9.1 配置过程

当 I2C 主设备重新发送 0xC1 命令后，将表 3-2-1 中的 reboot 设置为 0，重启 FPGA，假设 Slave Address 为 0x78，其时序示意图，如图 5-46 所示。

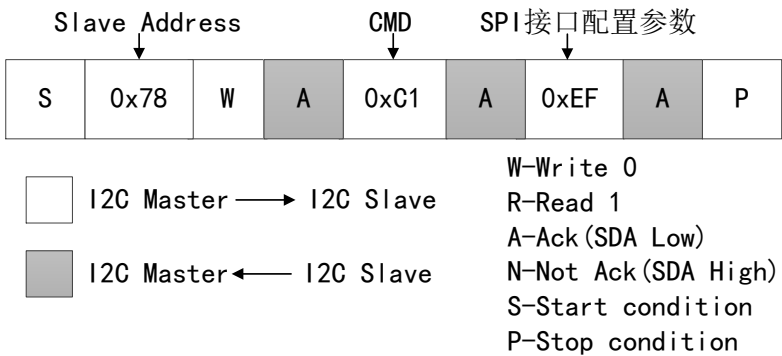


图 5-46 I2C 主设备重启 FPGA 的时序示意图

图 5-46 中，SPI 接口配置参数为 0xEF（OTHERS 设置为 1，REBOOT 设置为 0，CSN、DIRECTION、CPHA 以及 CPOL 设置为 1），I2C 主设备重启 FPGA 的时序图，如图 5-47 所示。

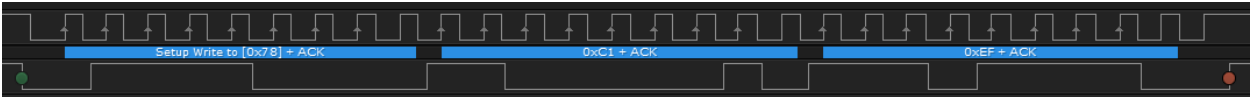


图 5-47 I2C 主设备重启 FPGA 的时序图

5.9.2 嵌入式函数 API 说明

```
void CfgCMD01_Reboot(void)
```

重启 FPGA 函数。

6 工程说明

6.1 FPGA 工程说明

该示例工程基于 EF2M45LG48 硬件平台进行测试验证，硬件连接如图 6-1 所示，src 文件夹包含 i2c_top.v 以及已加密的 i2c_to_spi_debounce_top_merge.enc.v 文件。后续可通过修改此工程进行移植，将 i2c_to_spi_debounce_top_merge.enc.v 嵌入到用户工程中。

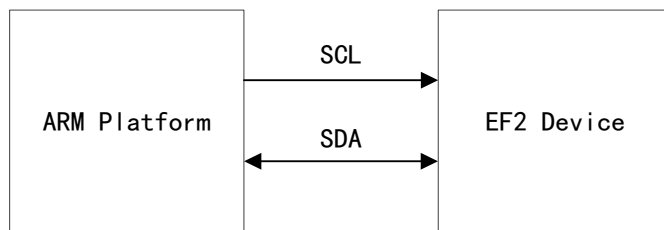


图 6-1 硬件连接

6.2 ARM 工程说明

该示例工程是基于 ARM CPU、Linux 操作系统测试验证，LinuxAPP 文件夹中包含 i2c_for_spi_softcore_cfg.c 和 i2c_for_spi_softcore_cfg.h，操作步骤如下

1. 打开待配置的文件(bin)，获取文件信息；
2. 打开 I2C 设备；
3. 通过 CMD 0xC1 配置 SPI 相关参数；
4. 擦除 Flash；
5. 读取 256 个字节，按 Page 进行烧写，待 bin 文件写完，结束；
6. 读取第一个 Page 数据；
7. 读取 Slave ID，此 ID 可在 RTL 代码中定义；
8. 读取 Flash ID；
9. 通过 CMD 0xC1 复位，重新加载位流文件；
10. 关闭 I2C 设备；

函数入口：i2cForSPI_SoftCore(char *write_file_name)

移植时，若是 Linux 系统，可根据 I2C 设备情况修改下述宏定义

```
#define CHIP        "/dev/i2c-0"
```



若是 MCU 环境，需要对打开/关闭 I2C、读写函数进行修改替换编写。

6.3 工程目录

i2c2spi_v2 文件夹层次如表 6-1 所示。其中，i2c_for_spi_softcore_cfg_add_read.c 文件增加 I2C 回读校验功能，支持 bit 文件直接转 bin 文件功能；bit2bin.c 文件包含 bit 转 bin 文件相关函数；xxx.h 文件为头文件。

表 6-1 i2c2spi_v2 文件夹层次表

父文件	子文件	包含材料
APUG063_I2C_Slave_Upgrade_FPGA _FLASH_Reference_Design_v1.2	constrs	board.adc
		iic2spi.sdc
	doc	APUG063_I2C 接口配置 FLASH 应用解决方案.pdf
	LinuxAPP	i2c_for_spi_softcore_cfg_add_read.c
		i2c_for_spi_softcore_cfg.h
		bit2bin.c
		bit2bin.h
	par	simulation
		iic2spi.al
	src	al_ip
		i2c_to_spi_debounce_top_merge.enc.v
		i2c_top.v



7 版本信息

日期	版本	修订记录
2022/10/17	1.0	首次发布中文版
2023/06/01	1.1	更新了图注和题注，工程文件使用的版本号为 TD5.6.2_71036
2025/06/05	1.2	1、调整了表 6-1 的文件夹层次结构； 2、将文中“设备地址（Slave Address，7 位）以及写信号（W，1 位）”描述修改为“设备地址（Slave Address，8bit 地址的高 7 位）以及写信号（W，8bit 地址的最低位）”
2025/08/20	1.3	1、增加了表 3-2 关于 HOLD_TIMING 的描述。
2025/09/04	1.4	1、更新了 6.3 节关于 .c 和 .h 文件描述。

版权所有© 2025 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其他方式授予任何知识产权许可；本文档仅为向用户提供使用器件的参考，协助用户正确地使用安路科技产品之用，其著作权归安路科技所有；本文档所展示的任何产品信息均不构成安路科技对所涉产品或服务作出任何明示或默示的声明或保证。

安路科技将不定期地对本文档进行更新、修订。用户如需获取最新版本的文档，可通过安路科技的官方



网站（网址为：<https://www.anlogic.com>）自行查询下载，也可联系安路科技的销售人员咨询获取。